

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 19

**СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С
БИБЛИОТЕКОЙ REQUESTS И МОДУЛЕМ URLLIB.REQUEST**

ЛАБОРАТОРНАЯ РАБОТА № 19.2

**СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С МОДУЛЕМ
URLLIB.REQUEST**

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____ от ____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для работы с модулем `urllib.request`

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Обмен данными с помощью модуля `urllib.request`

Модуль **`urllib.request`** предоставляет расширенные возможности для получения информации из Интернета. Поддерживаются автоматические перенаправления при получении заголовка **Location**, возможность аутентификации, обработка **cookies** и др.

Для выполнения запроса предназначена функция **`urlopen()`**.

Формат функции:

`urlopen (<URL-адрес или объект запроса>[, <Данные>][, <Тайм-аут>] [, cafile=None][, capath=None])`

В первом параметре задается полный **URL**-адрес или объект, возвращаемый конструктором класса **`Request`**. Запрос выполняется методом **GET**, если данные не указаны во втором параметре, и методом **POST** в противном случае. При передаче данных методом **POST** автоматически добавляется заголовок **Content-Type** со значением **`application/x-www-form-urlencoded`**.

Третий параметр задает максимальное время выполнения запроса в секундах. Метод возвращает объект класса **`HTTPRequest`**.

Этот класс поддерживает следующие методы и атрибуты:

– **`read ([<Количество байтов>])`** – считывает данные. Если параметр не указан, возвращается содержимое результата от текущей позиции указателя до конца. Если в качестве параметра указать число, то за каждый вызов будет возвращаться указанное количество байтов. Когда достигается конец, метод возвращает пустую строку.

Пример:

```
from urllib.request import urlopen
res = urlopen("http://mail.ru/testrobots.php")
print (res.read(34).decode("cp1251"))
```

`<!DOCTYPE html><html id="nojs" lang`

– **readline** ([<Количество байтов>]) – считывает одну строку при каждом вызове. При достижении конца возвращается пустая строка. Если в необязательном параметре указано число, то считывание будет выполняться до тех пор, пока не встретится символ новой строки (`\n`), символ конца или не будет прочитано указанное количество байтов. Иными словами, если количество символов в строке меньше значения параметра, будет считана одна строка, а не указанное количество байтов. Если количество байтов в строке больше, возвращается указанное количество байтов.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
print (res.readline(100).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lang="ru" class=" nonRetina"><head>
<!--MAIL.RU--><meta http-equiv="X-
```

– **readlines** ([<Количество байтов>]) – считывает весь результат в список. Каждый элемент списка будет содержать одну строку, включая символ перевода строки. Если параметр задан, считывается указанное количество байтов плюс фрагмент до конца строки. При достижении конца возвращается пустой список.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
res.readlines(1)
[<b><!DOCTYPE html><html id="nojs" lang="ru" class=" nonRetina"><head>
<!--MAIL.RU--><meta http-equiv="X-UA-Compatible"
content="IE=edge"/>&
lt;meta charset="utf-8"/>
<title>Mail.Ru:
. . . Пропуск фрагмента . . .
feed=http://news.mail.ru/rss/&id=5; cycle=1"/><script>\n'
```

– **__next__()** – считывает одну строку при каждом вызове. При достижении конца результата возбуждается исключение **StopIteration**. Благодаря методу **__next__()** можно перебирать результат построчно с помощью цикла **for**. Цикл **for** на каждой итерации будет автоматически вызывать метод **__next__()**.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
for line in res: print(line)
```

– **close()** – закрывает объект результата;

– **geturl()** – возвращает интернет-адрес полученного документа. Так как все перенаправления автоматически обрабатываются, интернет-адрес полученного документа может не совпадать с адресом, заданным первоначально;

– **info()** – возвращает объект, с помощью которого можно получить информацию о заголовках ответа сервера. Основные методы и атрибуты этого объекта были рассмотрены при изучении модуля **http.client**.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
info = res.info()
```

```

info.items()
[('Server', 'nginx/1.14.1'), ('Date', 'Sun, 06 Jan 2019 07:48:34 GMT'),
 ('Content-Type', 'text/html; charset=utf-8'), ('Content-Length', '299131'),
 ('Connection', 'close'),
 . . . Пропуск фрагмента . . .
 ('X-Content-Type-Options', 'nosniff'),
 ('Strict-Transport-Security', 'max-age=16070400')]
info.get("Content-Type")
text/html; charset=utf-8'
info.get_content_type(), info.get_content_charset()
('text/html', 'utf-8')
info.get_content_maintype(), info.get_content_subtype()
('text', 'html')
– code – содержит код возврата в виде числа;
– msg – содержит текстовый статус возврата:
res.code, res.msg
(200, 'OK')

```

Для примера выполним запросы методами GET и POST.

```

from urllib.request import urlopen
from urllib.parse import urlencode
data = urlencode ({"color": "Красный", "var" : 15}, encoding ="cp1251")
# Отправка данных методом GET
url = "http://mail.ru/testrobots.php?" + data
res = urlopen(url)
print (res.read(34).decode ("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()
# Отправка данных методом POST
url = "http://mail.ru/testrobots.php"
res = urlopen(url,data.encode("cp1251"))
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()

```

Если необходимо задать свое название робота и передать дополнительные заголовки, то следует создать экземпляр класса **Request** и передать его в функцию **urlopen()** вместо интернет-адреса. Конструктор класса **Request** имеет следующий формат:

```
Request (<URL-адрес> [, data=None][, headers={}][, origin_req_host=None][,
unverifiable=False][, method=None] )
```

В первом параметре указывается URL-адрес. Запрос выполняется методом **GET**, если данные не указаны во втором параметре, или методом **POST** в противном случае. При передаче данных методом **POST** автоматически добавляется заголовок **Content-Type** со значением **application/x-www-form-urlencoded**. Третий параметр задает заголовки запроса в виде словаря. Четвертый и пятый параметр используются для обработки **cookies**. Шестой параметр,

поддержка которого появилась в **Python 3.3**, указывает метод передачи данных в виде строки - например: "**GET**" или "**HEAD**". За дополнительной информацией по этим параметрам обращайтесь к документации. В качестве примера выполним запросы методами **GET** и **POST**.

```
from urllib.request import urlopen, Request
from urllib.parse import urlencode
headers = { "User-Agent": "MySpider/1.0",
"Accept": "text/html, text/plain, application/xml", "Accept-Language": "ru, ru-
RU",
"Accept-Charset": "windows-1251", "Referer": "/index.php" }
data = urlencode ({ "color": "Красный", "var" : 15}, encoding ="cp1251")
# Отправка данных методом GET
url = "http://mail.ru/testrobots.php?" + data
request = Request (url, headers=headers)
res = urlopen (request)
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
# Отправка данных методом POST
url = "http://mail.ru/testrobots.php"
request = Request(url, data.encode("cp1251"), headers=headers)
res = urlopen (request)
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()
```

4.2 Определение кодировки

Документы в Интернете могут быть представлены в различных кодировках. Чтобы документ был правильно обработан, необходимо знать его кодировку. Определить кодировку можно по заголовку **Content-Type** в заголовках ответа **Web**-сервера:

Content-Type: text/html; charset=utf-8

Кодировку **Web**-страницы можно также определить по значению параметра content тега **<meta>**, расположенного в разделе **HEAD**:

```
<meta http-equiv="Content-Type" content="text/html; charset=windows-1251">
```

Однако часто встречается ситуация, когда кодировка в ответе сервера не совпадает с кодировкой, указанной в теге **<meta>**, или же таковая вообще не указана. Определить кодировку документа в этом случае позволяет библиотека **chardet**. Установить ее можно из репозитория **PyPI** способом, описанным на [228 шаге](#). Открываем командную строку и набираем в ней команду:

```
c:\python34\scripts\pip install chardet
```

Через некоторое время библиотека будет установлена.

Для проверки установки запускаем редактор **IDLE** и в окне **Python Shell** выполняем следующий код:

```
import chardet
chardet.__version__
'3.0.4'
```

Определить кодировку строки позволяет функция **detect** (**<Последовательность байтов>**) из модуля **chardet**. В качестве результата она возвращает словарь с тремя элементами. Ключ **encoding** содержит название кодировки, ключ **language** - название языка, а ключ **confidence** - коэффициент точности определения в виде вещественного числа от 0 до 1. Пример определения кодировки приведен ниже.

```
import chardet
chardet.detect(bytes("Строка", "cp1251"))
{'encoding': 'windows-1251', 'language': 'Russian', 'confidence': 0.99}
chardet.detect(bytes("Строка", "utf-8"))
{'encoding': 'utf-8', 'language': '', 'confidence': 0.99}
```

Если файл имеет большой размер, то вместо считывания его целиком в строку и использования функции **detect()** можно воспользоваться классом **UniversalDetector**. В этом случае можно читать файл построчно и передавать текущую строку методу **feed()**. Если определение кодировки прошло успешно, атрибут **done** будет иметь значение **True**. Это условие можно использовать для выхода из цикла. После окончания проверки следует вызвать метод **close()**. Получить результат определения кодировки позволяет атрибут **result**. Очистить результат и подготовить объект к дальнейшему определению кодировки можно с помощью метода **reset()**. Пример использования класса **UniversalDetector** приведен ниже.

```
from chardet.universaldetector import UniversalDetector
ud = UniversalDetector() # Создаем объект
for line in open("file.txt", "rb"):
    ud.feed(line)        # Передаем текущую строку
    if ud.done: break    # Прерываем цикл, если done == True
ud.close()              # Закрываем объект
print(ud.result)        # Выводим результат
Результат работы приложения:
{'confidence': 0.8726928676836571, 'encoding': 'MacCyrillic', 'language':
'Russian'}
```

Как показали тесты, при использовании кодировки **Windows-1251** файл просматривается полностью, и определение кодировки файла, содержащего 6500 строк, занимает почти секунду. Если сменить кодировку файла на **UTF-8** без **BOM**, то время определения увеличивается до 5 секунд.

4.3 Пример решения задачи на языке программирования Python

Задача. Напишите программу на Python с использованием модуля **urllib.request**, которая загружает HTML-код веб-страницы по указанному URL-адресу, преобразует полученные данные в строку и выводит следующие

результаты: первые 100 символов страницы, количество символов во всём HTML-коде, а также сообщение об успешном получении данных.

Решение задачи:

```
import urllib.request
url = input("Введите URL: ")
response = urllib.request.urlopen(url)
html_bytes = response.read()
html_text = html_bytes.decode("utf-8", errors="ignore")
print("Первые 100 символов:")
print(html_text[:100])
print("\nОбщее количество символов:", len(html_text))
print("\nДанные получены успешно")
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите программу, которая считывает данные с веб-страницы <https://example.com> и выводит количество символов в полученном тексте.

2 Напишите программу, которая получает HTML-код страницы <https://example.com> и выводит только первые 200 символов.

3 Напишите программу, которая получает данные с веб-страницы и выводит их тип данных до декодирования.

4 Напишите программу, которая считывает данные с URL-адреса и выводит результат в виде строки.

5 Напишите программу, которая загружает страницу <https://example.com> и выводит длину HTML-кода.

6 Напишите программу, которая открывает веб-страницу и выводит результат построчно.

7 Напишите программу, которая загружает страницу и выводит последние 100 символов полученного текста.

8 Напишите программу, которая считывает данные с URL-адреса и выводит их в консоль без форматирования.

9 Напишите программу, которая получает HTML-код веб-страницы и сохраняет его в файл с произвольным именем.

10 Напишите программу, которая загружает страницу и выводит сообщение об успешном получении данных.

11 Напишите программу, которая считывает данные с URL-адреса и выводит первые 5 строк HTML-кода.

12 Напишите программу, которая получает содержимое веб-страницы и подсчитывает количество строк в HTML-коде.

13 Напишите программу, которая получает содержимое веб-страницы и выводит сообщение «Данные получены»

14 Напишите программу, которая с помощью `urllib.request` проверяет, доступен ли веб-ресурс, и выводит код состояния ответа сервера.

15 Напишите программу, которая считывает данные с веб-страницы и выводит количество слов в полученном HTML-коде.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите модуль `urllib.request` в языке программирования Python.

7.2 Приведите формат функции `request()` в языке программирования Python.

7.3 Охарактеризуйте основные методы и атрибуты класса `HTTPRequest` в языке программирования Python.

Список используемых источников

1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.